EECS370 Mid

Intro

Moore's Law: # of transistors in a chip doubles every 2 years

Denard Scaling: power density stays constant (In reality, leakage of power occur)

ISA

LC2K

* 32-bit (each reg)
* 8 registers ⇒ PC = PC+1
* 65536 words of memory (word addressable)
* add 000; nand 001

24-22	opcode
21-19	regA
18-16	regB
2-0	destR.

* lw 010; sw 011; beq 100

24-22	opcode
21-19	regA
18-16	regB
15-0	offset

if (regA = regB) PC=PC+1 +offset

* jalr 101

24-22	opcode
21-19	regA
18-16	regB

reg B = PC+1
PC = reg A

* halt 110 noop 111

24-22 opcode

ARM arithmetic ⇒ PC = PC+4

add (adc) dest, src1, src2imm
sub (sbc) dest, src1, src2imm
mul dest, src1, src2
and, orr, eor dest, src1, src2imm
mov dest, src1 (, LSL/LSR src2imm)
cmp src1, src2imm

* supports 12-bit offset

save/load

ldr, ldrh, ldrb.
str, strh, strb.
*加"s"表示 sign extend.
e.g. str r3, [r4, #1000]

reg
0A0B0C0D

0A		0D	a
0B		0C	a+1
0C		0B	a+2
0D		0A	a+3

Big Endian
Little Endian

branch

e.g. cmp r1, r2
beq label. <offset

Conditions:
eq. ne. gt. lt. gt. le. X
bl: branch & store PC in link register

Converting C to assembly

Data Layout

*p 4 byte
double 8 byte
float 4 byte
int 4 byte
short 2 byte
char 1 byte

* starting address & size of struct decided by largest field.

ARM memory map

stack ↓
↑ heap ← dynamic
static ← globals & static locals
text ← instructions

0x7fffffff
0x08048000

ARM registers

r0	return value
r1-r3	parameter, caller~
r4-r10	callee~
r11	frame ptr
r12	
r13	stack ptr
r14	link reg
r15	= pc

Function Call

caller saved:
if "live" across func call
callee saved:
if overwritten

e.g.
sub r13, r13, #16 allocate
str r4 [r13, #8] callee sv
str r14 [r13, #12] sv return add.
...
mov r0, #0
ldr r4 [r13, #8] restore
ldr r14, [r13, #12] restore
add r13, r13, #16 deallocate
mov r15, r14 return

incoming parameters
return address
~~previous frame ptr~~ [r11, #0]
callee saved regs [r11, #-4]
local variables
spilled registers
caller saved regs
outgoing parameters [r13, #0]

Numbers

Negative numbers: 2's complement
bitwise negation & plus one. ↔

Floating point: IEEE754 (24 bit precision)
Sign bit: 0-positive; 1-negative
mantissa: 23 bits after the decimal point
exponent: biased 127 encoding 8 bits

e.g. -127 → 00000000 1 → 10000000
-126 → 00000001 2 → 10000001
0 → 01111111 128 → 11111111

e.g. $10.625_{10} \Rightarrow 1010.101_2 \Rightarrow 1.010101 \times 2^3$

⇒ | 1 | 10000101 | 01011001000000000000000 |

sign	exponent	mantissa
1 bit	8 bits	23 bits

.c —compiler→ .as —assembler→ .o —linker→ .out —loader→
.o ↗

Object code format

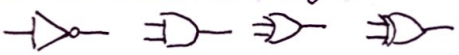
Header	size of text, data.
Text	machine code
Data	initidized global/static
Symbol Table	function, global/static
Relocation Table	address, constants (string)
Debug Info	function call (bl, jalr) global/static

* pc-relative addressing e.g. beq. never relocate
* absolute address. e.g. mov r15, x.
  external reference e.g. bl.
  data reference e.g. movw/movt
  } always relocate

Foundations of Processor Design

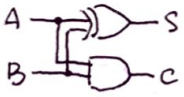
Combinational Logic

NMOS

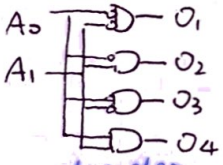


not and or xor ($a \oplus b = a'b + ab'$)

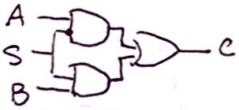
* nand can be used to implement any gates



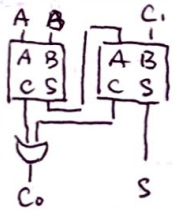
half-adder



decoder

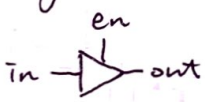


mux



full adder

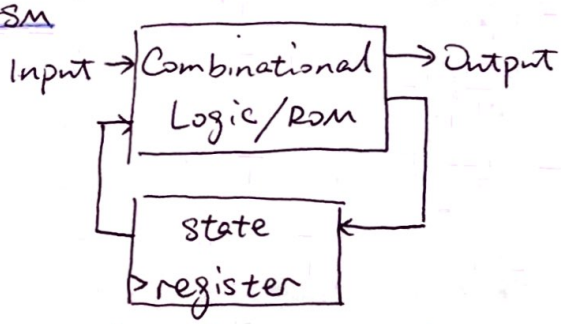
*gate 除 inverter 都算作 delay



Memory elements

SR Latch → D Latch → DFF

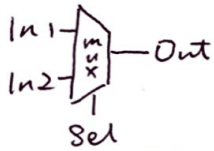
FSM

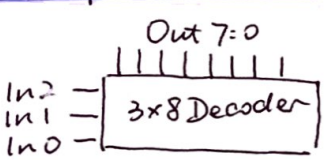


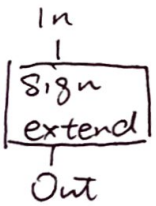
Moore: based on state only

Mealy: based on state & input
output function

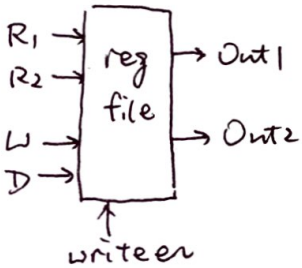
LC2K Datapath Implementation

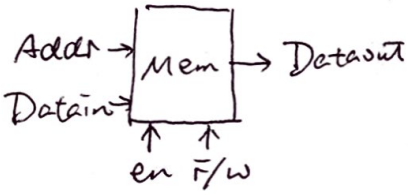




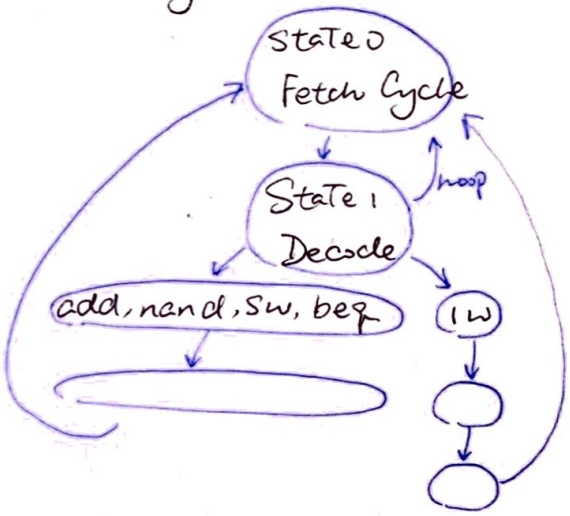


func (0: and; 1: nand)





Single Cycle
Multicycle



EECS370 Final

pipeline

IRON LAW: $t = \#instr \times CPI \times T$

processor
- single-cycle
- multi-cycle
- pipeline

multicycle

Fetch → Decode → ALU Execution → Register Write (add, nand) / Memory Access (sw, lw) / PC += offset (beq)

single cycle

- mem access ×2
- reg read + write
- ALU

pipeline

① IF	② ID	③ EX	④ MEM	⑤ WB
instr fetch	reg read	execution	branch resolved	mem access done

(lw available for forwading)

hazard
- data ~
- control ~
- structural ~

Data Hazard

*occur within 2 instruction window if internal forwarding reg.

Detect & Stall

add/nand/lw 后 noop ×2

Detect & Forward

从各reg forward 到 EX

add/nand 不用 noop

lw 后 noop ×1 (除 lw 了 ⑥, 特例)

sw × ⑥ ×

Control Hazard (beq)

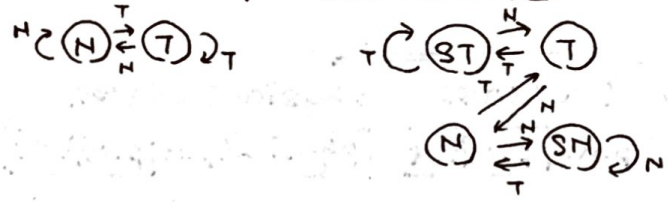
Detect & Stall 3 noop

Speculate & Squash 3 noop if wrong

(instr left in IE, noop sent to ID.EX.MEM)

branch prediction

- static
  - always taken / not taken
  - BTFN (backward T, forward N)
- dynamic
  - last time prediction
  - 2-bit counter ~



cache

- temporal locality – LRU/LFU/FIFO
- spacial locality

avg latency = cache latency + memory latency × miss rate

- write through
- write back (dirty bit)

allocate on write

- fully-associative #set = 1
- directly-mapped #way = 1
- set-associative

address

tag	set index	block offset

cache size = blocksize × #blocks (always 2^n)

#sets × #ways

addr = tag + bit blockoffset + bit set idx

overhead storage = tag + valid + dirty

3Cs cache misses

- compulsory ~ : cold start
- capacity ~ : hit if larger cache
- conflict ~ : hit if higher associativity

* simulate with ∞ cache
* simulate with fully associative cache
* simulate with intended cache.

virtual memory

* transparency * protection

cache (SRAM)
main memory (DRAM)
disk

single level page table

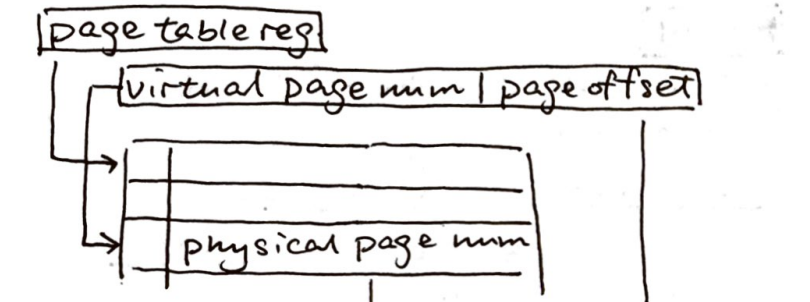
virtual address: | virtual p.n. | p.offset |

physical address: | physical p.n | p.offset |

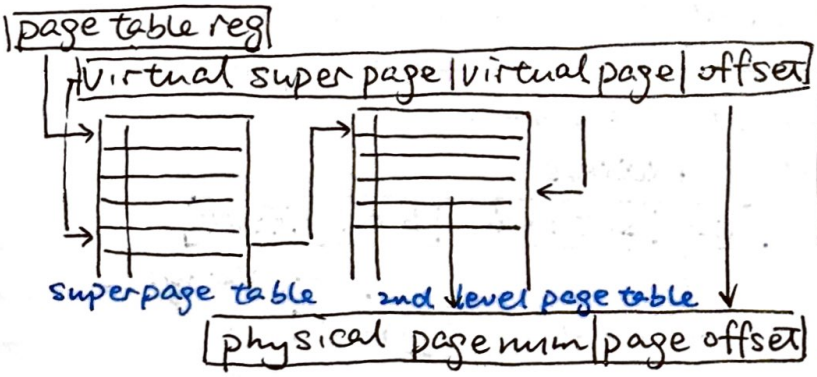
0	0000	8	1000
1	0001	9	1001
2	0010	A	1010
3	0011	B	1011
4	0100	C	1100
5	0101	D	1101
6	0110	E	1110
7	0111	F	1111

$2^0=1$	$2^6=64$
$2^1=2$	$2^7=128$
$2^2=4$	$2^8=256$
$2^3=8$	$2^9=512$
$2^4=16$	$2^{10}=1024$
$2^5=32$	

K: 2^{10} M: 2^{20} G: 2^{30}



multi level page table (size ∝ mem used)



TLB

V	tag	physical page

cache

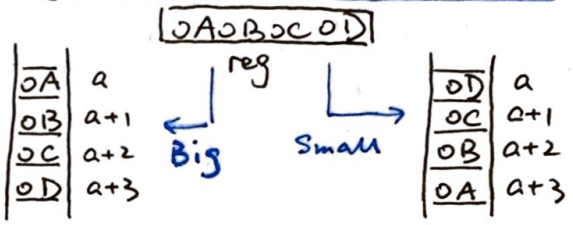
- physically addressed (slow)
- virtually addressed (complex)
  - * perform TLB lookup at the same time as the cache tag compare

problems
- * same v. addr → diff p. addr
- * diff v addr → same p. addr

⇒ cache flushed between context switch

processID as part of tag

Big Endian vs. Small Endian



Data Layout

- *p 4B
- double 8B
- float 4B
- int 4B
- short 2B
- char 1B

Object Code

- Header
- Text
- Data
- Symbol Table
- Debug Info

LC2K

- *32bit reg ×8
- *65536 words mem
- *1 word = 4 byte

Numbers

Negative: 2's Complement

bitwise neg +1 ↔

Floating

sign: 0 +; 1 −

mantissa: 23 bits after "."

exponent: biased 127

encoding 8 bits

e.g. −127 → 00000000

−126 → 00000001

0 → 01111111

1 → 10000000

2 → 10000001

128 → 11111111

e.g. $10.625_{10} \Rightarrow 1010.101_2$

$\Rightarrow 1.010101 \times 2^3$

⇒ 1 | 10000101 | 01011001...